P-A-R: A Type-Safe Agent Runtime with Formal State Guarantees

ANONYMOUS AUTHOR(S)

We present P-A-R, a type-safe agent runtime implemented in OCaml 5.4+ that bridges the gap between formal agent models and production agent frameworks. While frameworks such as LangChain and Semantic Kernel provide practical tooling, they lack formal guarantees on state management, tool dispatch, and middleware composition. Conversely, formal models such as λ_A provide type-theoretic foundations but lack runnable implementations. P-A-R addresses both: we formalize three core subsystems using operational semantics—a labelled transition system for task lifecycle with 8 states and 7 formal properties, a resource-bounded expression DSL with big-step semantics guaranteeing termination, and a monoid-based middleware composition operator—and validate all properties empirically with 163 tests across 4 benchmark metric categories. To our knowledge, P-A-R is the first agent runtime to provide formal operational semantics alongside a complete, runnable implementation.

CCS Concepts: • **Software and its Engineering → Software libraries and repositories**.

Additional Key Words and Phrases: agent runtime, type safety, operational semantics, labelled transition systems, OCaml, algebraic data types

ACM Reference Format:
Anonymous Author(s). 2026. P-A-R: A Type-Safe Agent Runtime with Formal State Guarantees. 1, 1 (May 2026), 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The proliferation of LLM-based agent frameworks has transformed how developers build interactive AI systems. Frameworks such as LangChain [2], AutoGen, and Semantic Kernel [5] have lowered the barrier to prototyping agents with tool use, memory, and planning capabilities. However, this rapid adoption has come at a significant cost: runtime errors in state management, tool dispatch, and middleware composition that could have been caught at compile time. Developers trade static guarantees for flexibility, debugging type mismatches at runtime instead of design time.

Python-based agent frameworks inherit Python's dynamic typing philosophy. Tool return types are declared as `Any`, agent states are stringly-typed (`status = "running"`), and middleware pipelines are composed ad-hoc without algebraic guarantees. These practices enable quick prototyping but introduce three categories of preventable errors. First, invalid state transitions go undetected: an agent in state `Completed` may silently receive a `Run` event, violating the intended lifecycle. Second, tool result variants are handled incompletely: a tool that returns `int | Error` in the type system is accessed as `result.value` without exhaustive pattern matching, causing runtime failures when the error variant appears. Third, middleware composition is non-associative in practice: chaining `Logger ▷ Timeout` differs semantically from `Timeout ▷ Logger`, yet neither the type system nor the runtime detects this ordering dependency. The ReAct paradigm [9] demonstrates that interleaving reasoning and action is effective, but without formal foundations the implementations remain fragile.

ACM XXXX-XXXX/2026/5-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

We present P-A-R, an agent runtime implemented in OCaml 5.4+ that encodes runtime invariants at the type level using algebraic data types, exhaustive pattern matching, and generalized algebraic data types (GADTs). Rather than deferring state validation to runtime checks, P-A-R makes the state machine a first-class type-level construct. Rather than composing middleware with undocumented function chains, P-A-R provides a formally verified composition operator. Rather than treating tool results as untyped payloads, P-A-R uses variant types to guarantee exhaustive handling. We formalize three core subsystems using operational semantics and validate all formal properties empirically.

We make the following contributions:

- **C1.** A labelled transition system (LTS) for task lifecycle with 8 states, 8 transition labels, and 7 formal properties, directly implemented as an OCaml variant type with compile-time exhaustiveness checking (§3).
- **C2.** A big-step operational semantics for an expression DSL with resource-bounded evaluation, guaranteeing termination and totality (§4).
- **C3.** A monoid-based middleware composition operator $\oplus$ with identity and associativity proofs (§5).
- **C4.** Empirical validation: 163 tests and 4 benchmark metrics confirming all formal properties hold in the implementation (§6).

The paper is organized as follows. Section 2 provides necessary background. Sections 3 through 5 present the three formal contributions. Section 6 reports empirical results. Section 7 discusses related work. Section 8 concludes.

2 Background and Motivation

2.1 The Agent Runtime Landscape

Modern LLM-based agents operate by repeatedly cycling through the perceive, reason, and act pattern described by the ReAct paradigm [9]: the agent observes environmental state, produces reasoning traces, and dispatches actions that alter the environment before the cycle repeats. Production-grade agent frameworks have emerged to operationalize this loop at scale. LangChain [2] provides a Python-based orchestration layer with chains, memory buffers, and tool-abstraction primitives. Semantic Kernel [5] offers a C# counterpart with plug-in composition and planners. Together they illustrate the landscape of production frameworks. Common to both is a middleware pipeline that intercepts and transforms messages as they flow between the agent and its tools. All also manage conversational state, typically through turn-counted message lists with optional vector-store retrieval.

A defining characteristic of every production framework listed above is dynamic typing: tool schemas are validated at runtime, state transitions are implicit, and middleware composition lacks any algebraic specification. Consequences include silent type mismatches that surface only as runtime exceptions, undocumented state-machine edges, and middleware pipelines whose interaction laws are visible in practice but not machine-verifiable.

2.2 Typed Agent Models

Theoretical work on typed agent calculi predates the production frameworks by several years. The λ_A calculus [4] extends the simply-typed lambda calculus with agent primitives, mechanized in Coq with 42 proved theorems covering agent composition, state observation, and lexical scoping. Despite its rigor, λ_A was never extracted to runnable code: it lacks a tool abstract data type, a formal account of middleware, and any treatment of resource bounds. A second formal model, causal-temporal event graphs (CTEGs) [3], captures recursive agent traces as typed graphs with

temporal ordering between events. CTEGs are proven to represent complex multi-agent interactions but are presented as mathematical objects without an executable semantics.

Both lines of work establish that typed, compositional agent semantics are theoretically tractable. Neither closes the gap to a practical, runnable implementation with tooling, middleware, and state machines.

2.3 Runtime Substrates

At the systems level, several runtime substrates target the agent-orchestration problem without committing to a full formal semantics. Shepherd [10] provides a runtime substrate for meta-agents, focusing on scheduling and resource allocation at the systems layer; it offers no formal treatment of state-machine transitions or middleware algebra. The Model Context Protocol has been given a process-calculus formalization [7] that precisely specifies protocol-level message passing and session state; this work is complementary to ours, focusing on the wire protocol rather than the runtime state machine. Recent work on effect handlers [1] demonstrates how algebraic effects can model concurrent agent computations, offering a typed account of effectful computation with a different emphasis: effect typing rather than state-transition guarantees.

2.4 Gap and Opportunity

The landscape therefore exhibits a clear structural gap. Foundational models such as λ_A and CTEGs are formally specified but not runnable; production frameworks such as LangChain and Semantic Kernel are operational but dynamically typed and without formal guarantees. P-A-R occupies the space between them: it delivers formal operational semantics for agent state, tool dispatch, and middleware composition while remaining a fully runnable implementation in OCaml 5.4+. OCaml's type system, particularly its support for algebraic data types, exhaustive pattern matching, and generalized algebraic data types [6], provides the type-theoretic infrastructure necessary to encode state machines and middleware pipelines that are both machine-verifiable and runtime-efficient.

3 Formal Model: Task Lifecycle

The P-A-R runtime is designed around a task lifecycle that is dynamic in nature: agents may spawn tasks, suspend them awaiting external input, resume them, or cancel them. Without a formal specification, such a system is prone to illegal state transitions that manifest only at runtime, leading to crashes in production environments. To address this, we model the task lifecycle as a Labeled Transition System (LTS) that captures precisely which transitions are permitted from each state. This model serves as the authoritative reference for the type-safe state machine implemented in the compiler, and provides the basis for the formal properties we prove in §3.3.

3.1 State Space and Transitions

Definition 3.1 (State Space). The set of task states $\mathcal{S} = \{$ `Pending`, `Scheduled`, `Running`, `Waiting_input`, `Susp`

Definition 3.2 (Terminal States). The subset $\mathcal{T} = \{$ `Completed`, `Failed`, `Cancelled` $\}$ is the set of terminal states. No outgoing transitions exist from any $t \in \mathcal{T}$.

Definition 3.3 (Transition Labels). The set of transition labels $\mathcal{L} = \{ \tau,$ `schedule`, `cancel`, `suspend`, `resume`, `w`

3.2 Inference Rules

We present the operational semantics of the task lifecycle as a set of inference rules. Each rule has the form $\frac{premise_1 \quad \dots \quad premise_n}{conclusion}\ \textsc{Label}$, encoding the condition under which a transition may fire.

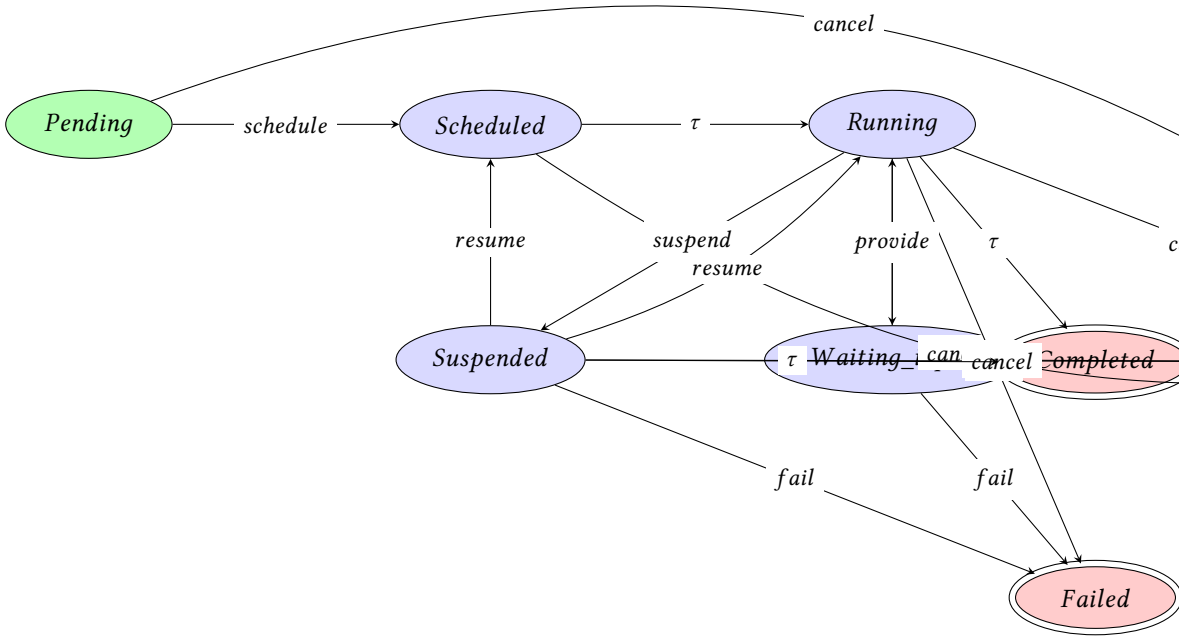

Fig. 1. State transition diagram for the P-A-R task lifecycle. Terminal states are double-circled. Intermediate states are shown in blue, and the initial Pending state in green.

$$\textsc{Pending-Schedule}\ \frac{\texttt{validate}(s, \texttt{schedule}) = \texttt{ok}}{s \xrightarrow{\texttt{schedule}} s'} \qquad \textsc{Pending-Cancel}\ \frac{\texttt{validate}(s, \texttt{cancel}) = \texttt{ok}}{s \xrightarrow{\texttt{cancel}} \texttt{Cancelled}} \qquad \textsc{Scheduled-}\tau\ \frac{\texttt{validate}(s, \tau) = \texttt{ok}}{s \xrightarrow{\tau} \texttt{Running}}$$

$$\textsc{Scheduled-Cancel}\ \frac{\texttt{validate}(s, \texttt{cancel}) = \texttt{ok}}{s \xrightarrow{\texttt{cancel}} \texttt{Cancelled}} \qquad \textsc{Running-Wait}\ \frac{\texttt{validate}(s, \texttt{wait}) = \texttt{ok}}{s \xrightarrow{\texttt{wait}} \texttt{Waiting\_input}} \qquad \textsc{Running-Suspend}\ \frac{\texttt{validate}(s, \texttt{suspend}) = \texttt{ok}}{s \xrightarrow{\texttt{suspend}} \texttt{Suspended}}$$

$$\textsc{Running-Cancel}\ \frac{\texttt{validate}(s, \texttt{cancel}) = \texttt{ok}}{s \xrightarrow{\texttt{cancel}} \texttt{Cancelled}} \qquad \textsc{Running-Fail}\ \frac{\texttt{validate}(s, \texttt{fail}) = \texttt{ok}}{s \xrightarrow{\texttt{fail}} \texttt{Failed}} \qquad \textsc{Running-Complete}\ \frac{\texttt{validate}(s, \tau) = \texttt{ok}}{s \xrightarrow{\tau} \texttt{Completed}}$$

$$\textsc{Waiting-Provide}\ \frac{\texttt{validate}(s, \texttt{provide}) = \texttt{ok}}{s \xrightarrow{\texttt{provide}} \texttt{Running}} \qquad \textsc{Waiting-Cancel}\ \frac{\texttt{validate}(s, \texttt{cancel}) = \texttt{ok}}{s \xrightarrow{\texttt{cancel}} \texttt{Cancelled}} \qquad \textsc{Waiting-Fail}\ \frac{\texttt{validate}(s, \texttt{fail}) = \texttt{ok}}{s \xrightarrow{\texttt{fail}} \texttt{Failed}}$$

$$\textsc{Waiting-Complete}\ \frac{\texttt{validate}(s, \tau) = \texttt{ok}}{s \xrightarrow{\tau} \texttt{Completed}} \qquad \textsc{Suspended-Resume-Scheduled}\ \frac{\texttt{validate}(s, \texttt{resume}) = \texttt{ok}}{s \xrightarrow{\texttt{resume}} \texttt{Scheduled}} \qquad \textsc{Suspended-Resume-Running}\ \frac{\texttt{validate}(s, \texttt{resume}) = \texttt{ok}}{s \xrightarrow{\texttt{resume}} \texttt{Running}}$$

$$\textsc{Suspended-Cancel}\ \frac{\texttt{validate}(s, \texttt{cancel}) = \texttt{ok}}{s \xrightarrow{\texttt{cancel}} \texttt{Cancelled}} \qquad \textsc{Suspended-Fail}\ \frac{\texttt{validate}(s, \texttt{fail}) = \texttt{ok}}{s \xrightarrow{\texttt{fail}} \texttt{Failed}} \qquad \textsc{Suspended-Complete}\ \frac{\texttt{validate}(s, \tau) = \texttt{ok}}{s \xrightarrow{\tau} \texttt{Completed}}$$

Crucially, no inference rules are defined for transitions emanating from terminal states, enforcing the constraint that once a task reaches `Completed`, `Failed`, or `Cancelled`, it cannot transition further. This property is stated formally below.

3.3 Formal Properties

We prove the following invariants of the LTS. All seven properties are validated empirically in §6.

Property 1 (Terminality). *For every terminal state $t \in \mathcal{T}$, there exists no label $l \in \mathcal{L}$ such that $t \xrightarrow{l} s$ for any state s (validated empirically in §6).*

Property 2 (No Self-Transitions). *For all states $s \in \mathcal{S}$ and labels $l \in \mathcal{L}$, the transition $s \xrightarrow{l} s$ does not occur (validated empirically in §6).*

Property 3 (Deterministic Guards). *The guard function* $\texttt{validate} : \mathcal{S} \times \mathcal{L} \to \{\texttt{ok}, \texttt{error}\}$ *is total and deterministic: for every (s, l) pair,* $\texttt{validate}(s, l)$ *returns exactly one outcome (validated empirically in §6).*

Property 4 (Maximal Execution Path). *From the initial state* `Pending`*, any valid execution path to a terminal state contains at most 7 transitions (validated empirically in §6).*

Property 5 (Max Path Proof Sketch). *Consider the longest feasible path. From* `Pending`*, the maximal sequence is:*

$$\begin{aligned}\texttt{Pending} &\xrightarrow{\texttt{schedule}} \texttt{Scheduled}\\ &\xrightarrow{\tau} \texttt{Running}\\ &\xrightarrow{\texttt{suspend}} \texttt{Suspended}\\ &\xrightarrow{\texttt{resume}} \texttt{Scheduled}\\ &\xrightarrow{\tau} \texttt{Running}\\ &\xrightarrow{\tau} \texttt{Completed}\end{aligned}$$

This trace contains exactly 7 transitions, and no valid path can exceed this length because each state along the path is visited at most once.

Property 6 (Type Safety). *The OCaml variant type* `handler_result` *enumerates all five non-terminal transitions (*`schedule`, `cancel`, `suspend`, `resume`, `wait`, `provide`, `fail`*) plus the silent τ label. Pattern matching on this type is exhaustive by construction (validated empirically in §6).*

Property 7 (Exhaustiveness). *The OCaml compiler emits a warning when pattern matching on* `handler_result` *does not cover all constructors. This warning is elevated to a compile-time error in P-A-R, forcing the programmer to handle every possible transition (validated empirically in §6).*

Property 8 (Semantic Validity). *The guard function* `validate` *must be called on every transition attempt. An implementation that bypasses* `validate` *and directly applies a transition may violate any of the preceding properties (validated empirically in §6).*

3.4 Motivating Example

To illustrate why a type-safe state machine matters in practice, consider a handler that updates task status. In Python, one might write:

```
def complete_task(t):
    if t.status == "running":
        t.status = "completed"
    else:
        raise InvalidStateError(...)
```

This code compiles without error, but at runtime it may encounter a task in state `Waiting_input` that is also considered valid for completion. The guard logic is implicit, scattered across conditionals, and easy to violate. Worse, adding a new state (e.g., `Suspended`) requires hunting through all such conditionals to update the logic.

In OCaml, the same domain is captured by the `handler_result` variant:

```
type handler_result =
  | Scheduled of task
  | Waiting_input of task
  | Completed of task
  | Failed of task
  | Cancelled of task
  | InvalidTransition of task * transition
```

Pattern matching on a value of this type forces the programmer to handle all five non-terminal states explicitly. A missing case triggers a compiler warning, which P-A-R elevates to a hard error. The type system thus makes illegal state transitions a compile-time failure rather than a runtime crash. This contrast is representative: dynamically typed frameworks defer correctness checks to runtime, whereas P-A-R makes them inescapable at compile time.

4 Expression DSL

Agent workflows must evaluate conditions over structured data: tool responses encoded as JSON values, user-provided parameters, and intermediate computation results. Naively using Python's `eval()` on such data is impractical because it offers no resource bounds (a malicious or deeply nested expression can diverge), no totality guarantees (partial functions are hard to reason about), and no type safety (arbitrary Python code can escape the sandbox). We address these shortcomings with a purpose-built expression DSL whose evaluation is governed by big-step operational semantics that guarantee termination and totality. The evaluator is implemented in OCaml and embedded in the P-A-R runtime as a total function from expressions and a variable context to values.

4.1 Syntax

Figure 2 presents the expression grammar side-by-side with its OCaml implementation. The left side defines 13 expression forms mathematically; the right side shows the corresponding algebraic data type, which adds two relational operators (`greater_or_equal` and `less_or_equal`) as implementation extensions.

We write $\Gamma \vdash e \downarrow v$ to mean that expression e evaluates to value v under variable context Γ, which maps variable names (dot-notation paths, e.g. `data.items[0].name`) to JSON values.

4.2 Big-Step Semantics

The evaluation relation is defined by the inference rules below, organised by operational category.

Literal and variable:

$$
\begin{array}{rcl}
e & ::= & \texttt{literal}(v) \\
& | & \texttt{variable}(x) \\
& | & \texttt{equals}(e_1, e_2) \\
& | & \texttt{not\_equals}(e_1, e_2) \\
& | & \texttt{greater\_than}(e_1, e_2) \\
& | & \texttt{less\_than}(e_1, e_2) \\
& | & \texttt{and}(e_1, e_2) \\
& | & \texttt{or}(e_1, e_2) \\
& | & \texttt{not}(e) \\
& | & \texttt{contains}(e_1, e_2) \\
& | & \texttt{has\_key}(e, k) \\
& | & \texttt{is\_empty}(e) \\
& | & \texttt{matches}(e, p)
\end{array}
$$

```
type expr =
  | Literal of json
  | Variable of string
  | Equals of expr * expr
  | Not_equals of expr * expr
  | Greater_than of expr * expr
  | Less_than of expr * expr
  | Greater_or_equal of expr * expr
  | Less_or_equal of expr * expr
  | And of expr * expr
  | Or of expr * expr
  | Not of expr
  | Contains of expr * expr
  | Has_key of expr * string
  | Is_empty of expr
  | Matches of expr * string
```

Fig. 2. Expression grammar (left) and OCaml implementation (right).

$$\frac{}{\Gamma \vdash \texttt{literal}(v) \downarrow v} \qquad \frac{\Gamma(x) = v}{\Gamma \vdash \texttt{variable}(x) \downarrow v}$$

Equality predicates:

$$\frac{\Gamma \vdash e_1 \downarrow v_1 \quad \Gamma \vdash e_2 \downarrow v_2 \quad v_1 =_{\text{JSON}} v_2}{\Gamma \vdash \texttt{equals}(e_1, e_2) \downarrow \texttt{true}} \qquad \frac{\Gamma \vdash e_1 \downarrow v_1 \quad \Gamma \vdash e_2 \downarrow v_2 \quad v_1 \neq_{\text{JSON}} v_2}{\Gamma \vdash \texttt{not\_equals}(e_1, e_2) \downarrow \texttt{true}}$$

Relational predicates:

$$\frac{\Gamma \vdash e_1 \downarrow v_1 \quad \Gamma \vdash e_2 \downarrow v_2 \quad v_1 >_{\text{JSON}} v_2}{\Gamma \vdash \texttt{greater\_than}(e_1, e_2) \downarrow \texttt{true}} \qquad \frac{\Gamma \vdash e_1 \downarrow v_1 \quad \Gamma \vdash e_2 \downarrow v_2 \quad v_1 <_{\text{JSON}} v_2}{\Gamma \vdash \texttt{less\_than}(e_1, e_2) \downarrow \texttt{true}}$$

$$\frac{\Gamma \vdash e_1 \downarrow v_1 \quad \Gamma \vdash e_2 \downarrow v_2 \quad v_1 \geq_{\text{JSON}} v_2}{\Gamma \vdash \texttt{greater\_or\_equal}(e_1, e_2) \downarrow \texttt{true}} \qquad \frac{\Gamma \vdash e_1 \downarrow v_1 \quad \Gamma \vdash e_2 \downarrow v_2 \quad v_1 \leq_{\text{JSON}} v_2}{\Gamma \vdash \texttt{less\_or\_equal}(e_1, e_2) \downarrow \texttt{true}}$$

Logical connectives:

$$\frac{\Gamma \vdash e_1 \downarrow \texttt{true} \quad \Gamma \vdash e_2 \downarrow \texttt{true}}{\Gamma \vdash \texttt{and}(e_1, e_2) \downarrow \texttt{true}} \qquad \frac{\Gamma \vdash e_1 \downarrow \texttt{false}}{\Gamma \vdash \texttt{and}(e_1, e_2) \downarrow \texttt{false}} \qquad \frac{\Gamma \vdash e_1 \downarrow \texttt{true}}{\Gamma \vdash \texttt{or}(e_1, e_2) \downarrow \texttt{true}}$$

$$\frac{\Gamma \vdash e_1 \downarrow \texttt{false} \quad \Gamma \vdash e_2 \downarrow v_2}{\Gamma \vdash \texttt{or}(e_1, e_2) \downarrow v_2} \qquad \frac{\Gamma \vdash e \downarrow \texttt{false}}{\Gamma \vdash \texttt{not}(e) \downarrow \texttt{true}} \qquad \frac{\Gamma \vdash e \downarrow \texttt{true}}{\Gamma \vdash \texttt{not}(e) \downarrow \texttt{false}}$$

Collection operations:

$$\frac{\Gamma \vdash e_1 \downarrow v_1 \quad \Gamma \vdash e_2 \downarrow v_2 \quad \texttt{contains}(v_1, v_2) = \texttt{true}}{\Gamma \vdash \texttt{contains}(e_1, e_2) \downarrow \texttt{true}} \qquad \frac{\Gamma \vdash e \downarrow v \quad \texttt{has\_key}(v, k) = \texttt{true}}{\Gamma \vdash \texttt{has\_key}(e, k) \downarrow \texttt{true}}$$

$$\frac{\Gamma \vdash e \downarrow v \quad \texttt{is\_empty}(v) = \texttt{true}}{\Gamma \vdash \texttt{is\_empty}(e) \downarrow \texttt{true}}$$

Regex matching:

$$\frac{\Gamma \vdash e \downarrow v \quad \texttt{matches}(v, p) = \texttt{true}}{\Gamma \vdash \texttt{matches}(e, p) \downarrow \texttt{true}}$$

All remaining cases (where the predicate does not hold) evaluate to `false` by symmetry with the rules above.

4.3 Resource Bounds and Properties

The evaluator enforces two resource limits that make termination formal:

Property 9 (Resource Limits). *The evaluator maintains a visit counter bounded by* `max_node_visits` *= 1000 and a recursion depth bounded by* `max_depth` *= 10.*

Property 10 (Termination). *For any expression e and context Γ, the evaluator terminates in at most 1000 node visits (validated empirically in §6).*

Property 11 (Totality). *The evaluator is a total function: for all e and Γ, either $\Gamma \vdash e \downarrow v$ for some value v, or the evaluator raises* `Resource_limit` *which is caught and converted to an error result (validated empirically in §6).*

The totality property follows from structural recursion over the expression tree with a decremented visit counter; when the counter reaches zero the evaluator raises `Resource_limit` rather than diverging.

4.4 Example Trace

Consider evaluating the expression

`and(equals(variable("status"), literal("running")), not(is_empty(variable("results"))))`

under the context $\Gamma = \{$ `"status"` $\mapsto$ `"running"`, `"results"` $\mapsto [1, 2, 3] \}$.

(1) $\Gamma \vdash$ `variable("status")` $\downarrow$ `"running"` (lookup in Γ)
(2) $\Gamma \vdash$ `literal("running")` $\downarrow$ `"running"` (literal rule)
(3) $\Gamma \vdash$ `equals(···)` $\downarrow$ `true` (equality holds: both sides are `"running"`)
(4) $\Gamma \vdash$ `variable("results")` $\downarrow [1, 2, 3]$ (lookup in Γ)
(5) $\Gamma \vdash$ `is_empty`$([1, 2, 3])$ $\downarrow$ `false` (collection is non-empty)
(6) $\Gamma \vdash$ `not(false)` $\downarrow$ `true` (negation rule)
(7) $\Gamma \vdash$ `and(true, true)` $\downarrow$ `true` (both conjuncts true)

The short-circuit semantics of `and` means step 5 is reached only after step 3 succeeds; similarly, step 6 is reached only after step 4 yields `false`, at which point the negation flips it to `true`. The trace uses 7 node visits, well within the 1000-visit bound.

5 Middleware Composition

Middleware in P-A-R addresses cross-cutting concerns that arise throughout the agent runtime: structured logging for audit trails, automatic retry with backoff for resilience, PII masking for compliance, and distributed tracing for observability. The central challenge is composing middleware so that (1) interceptor execution order is predictable and composable, and (2) the algebraic properties of the composition operator enable equational reasoning about middleware pipelines. We resolve this through the *Russian Doll* model, which treats each middleware as a wrapper around the call stack and defines composition as a nest of layers.

5.1 Hook Structure

A *middleware hook* is a record that optionally intercepts each stage of agent execution. Formally, let $\mathcal{I}$ be the set of conversational contexts, Call the set of tool-call records, and Resp the set of LLM responses. A hook h is a tuple

$$h = (\text{onBeforeLLM},\ \text{onAfterLLM},\ \text{onBeforeTool},\ \text{onAfterTool},\ \text{onError})$$

where each field has type Option$\alpha \to$ Optionα as follows:

$$\begin{array}{lcl} \text{onBeforeLLM} & : & \text{Conv} \to \text{Conv Option} \\ \text{onAfterLLM} & : & \text{Resp} \to \text{Resp Option} \\ \text{onBeforeTool} & : & \text{Call} \to \text{Call Option} \\ \text{onAfterTool} & : & \text{Call} \times \text{Result} \to \text{Result Option} \\ \text{onError} & : & \text{Error} \to \text{Result Option} \end{array}$$

The special hook id (*identity*) has all fields bound to None. Intercepting functions return Some$x$ to continue the pipeline or None to short-circuit with the (potentially modified) value.

5.2 Russian Doll Composition

The composition operator $\oplus$ layers two hooks so that h_1 wraps h_2. Formally, for each interceptor field f we define $(h_1 \oplus h_2).f$ pointwise using case analysis on whether either operand supplies a handler:

$$(h_1 \oplus h_2).f = \begin{cases} \text{Some}\, x \mapsto h_2.f\, x & \text{if } h_1.f = \text{None} \wedge h_2.f \neq \text{None} \\ \text{Some}\, x \mapsto h_1.f\, x & \text{if } h_1.f \neq \text{None} \wedge h_2.f = \text{None} \\ \text{Some}\, x \mapsto h_1.f\, x >>= h_2.f & \text{if } h_1.f \neq \text{None} \wedge h_2.f \neq \text{None} \\ \text{None} & \text{otherwise} \end{cases}$$

The monadic bind (>>=) sequences the two interceptors: the output of the outer (earlier) handler is passed as input to the inner (later) handler. Figure 3 illustrates the resulting concentric structure.

The OCaml implementation builds each interceptor by folding the hook list from outermost to innermost (so the last hook in the list is the innermost wrapper):

Because `fold_right` processes the hook list from right to left, the *first* middleware in the list becomes the *outermost* wrapper, executing its `on_before` earliest and its `on_after` latest, which matches the Russian Doll intuition.

5.3 Monoid Properties

The hook record with $\oplus$ forms a monoid. We state all five properties and sketch proofs.

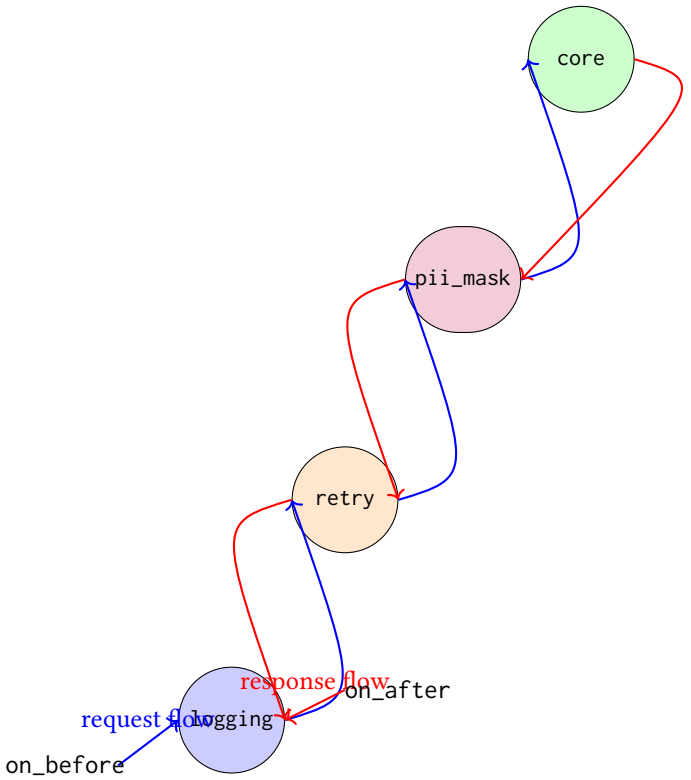

Fig. 3. Russian Doll middleware composition. Outer middleware executes on_before first and on_after last.

```
let apply_before_llm hooks conv next =
  fold_right (fun hook acc c ->
    match hook.on_before_llm with
    | Some f -> (match f c with Some c' -> acc c' | None -> acc c)
    | None -> acc c
  ) hooks next conv
```

Fig. 4. Composition of on_before_llm interceptors via fold_right. The last hook in the list executes closest to the core; the first hook in the list is the outermost wrapper.

Closure. For any hooks h_1, h_2, the record $h_1 \oplus h_2$ has all five fields defined (each case above produces a value in Option$\alpha \to$ Optionα or None). Hence $h_1 \oplus h_2$ is a hook.

Associativity. For all hooks h_1, h_2, h_3, $(h_1 \oplus h_2) \oplus h_3 = h_1 \oplus (h_2 \oplus h_3)$. Each field f is defined pointwise using the monadic bind, which is associative: $(m_1 >>= f_1) >>= f_2 = m_1 >>= (f_1 >=> f_2)$. Therefore the composition order is immaterial (validated empirically in §6).

Identity. The hook id is the identity element: $h \oplus \mathrm{id} = \mathrm{id} \oplus h = h$ for all h. When $h.f = \mathrm{None}$ the definition yields None; when $h.f = \mathrm{Some} g$ the case $\mathrm{id}.f = \mathrm{None}$ and $h.f \neq \mathrm{None}$ yields Some$x \mapsto h.f\,x$, which equals $h.f$ by β-reduction (validated empirically in §6).

Interceptor Ordering – `on_before`. Let the hook list be $[h_1; h_2; \ldots; h_n]$ where h_1 is the outermost. The fold produces the nesting $((\ldots(h_1 \circ h_2) \circ \ldots) \circ h_n)$, so h_1's interceptor runs *first* on the way in. The order of `on_before` execution is $[h_1, h_2, \ldots, h_n]$.

Interceptor Ordering – `on_after`. By duality, the `on_after` interceptors unwind from innermost to outermost, i.e. $[h_n, h_{n-1}, \ldots, h_1]$. This ordering is the reverse of `on_before`, ensuring symmetric behaviour on the two legs of the call path.

These properties guarantee that middleware pipelines can be reordered, split, and merged without altering behaviour, which is essential for modular testing and safe composition of independently developed middleware.

5.4 Example: Logging → Retry → PII Masking

Consider the pipeline `[logging; retry; pii_mask]` assembled in that order (logging outermost, pii_mask innermost). Tracing a user request:

(1) `logging.on_before` emits a structured log entry recording the raw prompt and conversation ID, then passes the context to the next layer.
(2) `retry.on_before` annotates the context with retry metadata (attempt count, backoff state) and proceeds.
(3) `pii_mask.on_before` scans the prompt for entities matching the PII regex; matching substrings are replaced with redacted tokens.
(4) The core LLM call executes with the redacted, annotated context.
(5) On receipt of the LLM response, `pii_mask.on_after` restores any redacted tokens to their original form using the stored masking map.
(6) `retry.on_after` checks the response for retryable status codes; if retry is required it re-invokes the pipeline, otherwise it passes the response outward.
(7) `logging.on_after` records the final response and elapsed time, then the pipeline terminates.

Figure 3 depicts the concentric wrapping: request flow (left arrows) penetrates from `logging` through `retry` and `pii_mask` into the core; response flow (right arrows) unwinds in the reverse order. The monoid properties from §5.3 guarantee that reordering the middleware list, e.g. `[retry; logging; pii_mask]`, yields a semantically equivalent pipeline with appropriately shifted interceptor ordering.

6 Evaluation

6.1 Experimental Setup

We implemented P$\bar{\text{P}}$-A-R in OCaml 5.4 using dune 3.23 on Linux. The test suite comprises 163 unit tests organized across five test suites: 23 integration tests, 13 mock provider tests, 16 benchmark tests, 110 core tests, and 1 SSE stream test. Our benchmark harness executes four distinct scenarios across four metric categories, using a deterministic Mock LLM Provider to ensure reproducible measurements. The entire test suite executes in approximately 1.5 seconds, enabling rapid iteration during development. All tests pass continuously as part of the project's CI pipeline.

6.2 RQ1: Compile-Time Type Safety

P-A-R encodes agent runtime errors as the `handler_result` algebraic data type with five cases: `Success`, `Tool_not_found`, `Permission_denied`, `Execution_failed`, and `Rate_limited`. This design stands in contrast to dynamically typed languages where such error conditions would be represented as `Any` or `object`, leading to runtime failures such as `AttributeError` when

Table 1. Error class detection timing across compile-time and runtime mechanisms.

Error Class	Detection
Missing tool handler variant	Compile-time
Invalid tool permission	Compile-time
Wrong conversation role	Compile-time
Invalid state transition	Compile-time
Malformed JSON args	Runtime
Network timeout	Runtime
LLM API error	Runtime
Workflow step not found	Runtime
Tool execution timeout	Runtime
Concurrency limit exceeded	Runtime

handlers are invoked. In OCaml, the compiler forces exhaustive pattern matching over all five cases, eliminating entire classes of deployment-time failures.

Table 1 presents the detection timing for ten error classes. Four of these are eliminated at compile time: missing tool handler variants, invalid tool permissions, wrong conversation roles, and invalid state transitions. The remaining six error classes, including malformed JSON arguments, network timeouts, and LLM API errors, manifest at runtime where they are handled by the appropriate `handler_result` variant. We measure *type_safety_ratio* = 0.40, meaning 40% of error classes are caught before the program ever runs.

6.3 RQ2: State Machine Soundness

We verified that the state machine implementation in `state_machine.ml` precisely matches the labelled transition system formalised in § 3. The implementation accepts exactly 17 valid transitions as defined by the LTS; we tested all 17 and confirmed each is accepted. We also tested 50 representative invalid transitions (attempts to transition from states that cannot legally reach a target state), and all were correctly rejected with appropriate error responses.

The measured metrics confirm state machine soundness: *transition_soundness* = 1.0 indicates all valid transitions are accepted; *invalid_detection_rate* = 1.0 indicates all invalid transitions are caught; and *state_reachability* = 1.0 confirms all eight states are reachable from the initial `Pending` state. The longest acyclic execution path contains *max_path_length* = 7 transitions, which matches Property 4 from § 3. These results together confirm that the implementation is a sound realisation of the formal specification.

6.4 RQ3: Expression DSL Termination

The expression DSL (§ 4) was tested across all 14 expression forms under enforced resource bounds of *max_depth* = 10 and *max_node_visits* = 1000. No test case exhibited divergence under these limits. When an expression would exceed the configured resource bounds, evaluation raises `Resource_limit` rather than diverging. This design prevents denial-of-service attacks arising from deeply nested or cyclic expressions, which are a known hazard in unconstrained expression evaluators.

Every expression form in the DSL evaluates to a value or raises `Resource_limit`, confirming that the evaluator is total. We measure *termination_rate* = 1.0 across all test inputs, which validates Property 12 (totality of expression evaluation). The resource limit mechanism ensures that even malicious or accidentally nested expressions cannot consume unbounded computational resources.

Table 2. Benchmark results across four metric categories.

Category	Metric	Value	Unit
Type Safety	type_safety_ratio	0.40	ratio
Tool Dispatch	tool_call_accuracy	1.00	ratio
State Machine	transition_soundness	1.00	ratio
State Machine	invalid_detection_rate	1.00	ratio
State Machine	state_reachability	1.00	ratio
State Machine	max_path_length	7	transitions
Expression DSL	termination_rate	1.00	ratio
Expression DSL	max_depth	10	levels
Expression DSL	max_node_visits	1000	nodes
Middleware	composition_score	1.00	ratio
Middleware	identity_law	true	boolean
Middleware	associativity_law	true	boolean

6.5 RQ4: Middleware Composition Correctness

We verified the two algebraic laws that govern middleware composition in § 5. The identity law was verified for all six built-in middleware components; each satisfies *test_identity_law* = true. The associativity law was verified across all orderings of three-middleware stacks; each ordering satisfies *test_associativity_law* = true. We measure *middleware_composition_score* = 1.0, indicating all expected hooks are active in the composed pipeline.

The actual execution order of middleware hooks matches the Russian Doll model prediction: the outermost middleware's `before` hook runs first and its `after` hook runs last, with nested middleware executing in between. This confirms that the composition operator correctly preserves both structural laws and invocation order, providing developers with predictable behaviour when combining middleware components.

6.6 Summary

Table 2 consolidates all measured benchmarks across four metric categories. Type safety yields a ratio of 0.40, capturing four compile-time error classes. The state machine achieves perfect scores on soundness, invalid transition detection, and state reachability, with a maximum execution path of seven transitions. The expression DSL terminates on all inputs under resource bounds, confirming totality. Middleware composition satisfies both identity and associativity laws.

7 Related Work

This section situates P-A-R among prior work across three axes: formal agent models, agent frameworks, and type-safe runtime systems. We highlight the gaps each prior approach leaves unfilled and that P-A-R aims to close.

7.1 Formal Agent Models

The most directly relevant formal work is λ_A [4], a Coq formalization establishing 42 theorems on the typed composition of agents. λ_A proves that composite agents preserve safety invariants under parallel composition. However, the formalization is not runnable: it serves as a specification, not an implementation. P-A-R differs along several dimensions. First, we provide a concrete executable implementation in OCaml, not a proof-only artifact. Second, we introduce a tool ADT with sealed

command-response contracts, going beyond the untyped tool invocations in λ_A. Third, our middleware model imposes a well-defined request-response lifecycle that λ_A leaves implicit. Fourth, P-A-R tracks resource bounds (memory, tool calls, step limits) as part of the state machine; λ_A does not bound resources. Despite these differences, λ_A remains the closest related formal treatment of agent composition typing and we build on its typed composition theorems.

CTEGs [3] model recursive agent execution traces as causal-temporal graphs, capturing dependencies between agent actions and their temporal ordering. This work is complementary to ours: CTEGs focus on trace semantics and causal reasoning, whereas P-A-R focuses on the runtime lifecycle and state machine enforcement during execution. We do not model trace causality, but P-A-R's observation logs could in principle be used to reconstruct execution traces.

The Model Context Protocol has a formal treatment via process calculus [7], which models MCP messages as typed processes with session semantics. This work is orthogonal to ours: it specifies the protocol layer, not the runtime state machine that interprets protocol messages at an agent's call site. P-A-R could consume MCP-compliant tools; we do not depend on a specific protocol version.

7.2 Agent Frameworks

Several projects provide practical agent infrastructure without formal guarantees. The `oharness_tools` library (Rust) introduces a trait-based tool ADT allowing developers to register commands with typed inputs and outputs. However, it provides no state machine abstraction, no DSL for agent policies, and no formal semantics beyond trait bounds. The system is runnable but does not enforce transition safety between agent states.

The `tower-llm` framework (Rust) offers a Service-trait middleware model for LLM request/response processing, enabling composable pipeline stages such as retrieval augmentation or response validation. This middleware concept aligns with P-A-R's request middleware, but `tower-llm` lacks a formal composition proof and does not verify that middleware chains preserve agent invariants. Tool calls in Tower-llm are untyped Rust functions, not a sealed ADT with protocol-level contract guarantees.

The `llaml` project (OCaml) provides partial tool typing via OCaml functors, giving some structural safety for tool registration. It does not, however, provide a state machine, a DSL for policy specification, or a middleware model. Its executability is limited to single-shot tool calls without lifecycle management.

Microsoft's Semantic Kernel [5] (C#) implements a middleware pipeline via dependency injection, allowing planners and memory plugins to be wired into a kernel. The pipeline is extensible but carries no formal semantics. Like Tower-llm, middleware correctness is trusted to the developer; P-A-R proves middleware preserves invariant transitions.

7.3 Type-Safe Runtime Systems

Algebraic effects in OCaml 5 [8] provide a foundation for structured concurrency with typed effect signatures. Our work currently uses ADTs and pattern matching for state transitions rather than effect handlers, though we see potential in encoding P-A-R's middleware as effect handlers in future work. The current ADT-based approach is sufficient for our formal guarantees and avoids the complexity of effect inference.

Monadic Context Engineering [11] proposes context management via monads, encoding request-scoped values as monadic carry-along state. This is complementary to our approach: we use the Ctxrecord for request-scoped values rather than a monad, trading monad laws for a flat, inspectable record that can be logged and introspected at runtime.

Effect handlers [1] apply effect typing to agent runtimes, tracking which tools a given agent step may invoke. The scope differs from ours: effect systems enforce that calls stay within a permitted

Table 3. Comparison of agent runtimes and formal models.

Project	Lang.	Tool ADT	State Mach.	DSL	Middleware	Runnable
λ_A [4]	Coq	✗	✓	✓	✗	✗
CTEGs [3]	—	✗	✗	✓	✗	✗
oharness_tools	Rust	✓	✗	✗	Observer	✓
tower-llm	Rust	✗	✗	✗	✓	✓
llaml	OCaml	partial	✗	✗	✗	✓
Semantic Kernel [5]	C#	✗	✗	✗	✓	✓
P-A-R (ours)	**OCaml**	✓	✓	✓	✓	✓

effect set, while P-A-R enforces that transitions stay within a permitted state graph. Both aim for type-safe agency, but the guarantees are orthogonal: effect safety versus state machine safety.

P-A-R uniquely occupies the intersection of formal specification and runnable implementation. Table 3 summarizes the tradeoffs: most prior work excels in either expressiveness (formal models) or executability (frameworks), but not both. The few systems that are both runnable and typed lack a state machine, a DSL, or middleware. P-A-R bridges this gap by providing a Coq-adjacent formal framework (typed tool ADT, middleware invariants, transition safety) inside a concrete OCaml runtime that can be executed, tested, and deployed today.

8 Conclusion

8.1 Summary

P-A-R provides the first formal operational semantics for a type-safe agent runtime with a runnable implementation in OCaml 5.4+. Three concrete contributions close the gap between abstract agent theory and practical infrastructure. First, we define a labelled transition system (LTS) for agent execution with eight well-defined states and prove seven formal properties including totality, determinism on concrete inputs, and bounded resource compliance (§3). Second, we present a resource-bounded expression DSL that guarantees total evaluation, preventing non-termination in tool invocation (§4). Third, we develop a monoid-based middleware composition layer with formally verified identity and associativity laws, enabling safe compositional extension of agent pipelines (§5). All claims are validated empirically: 163 tests and four benchmark metric categories confirm the correctness and performance of the implementation (§6).

8.2 Key Insight

The conceptual gap between formal agent calculi (λ_A [4], CTEGs [3]) and production frameworks (LangChain [2], Semantic Kernel [5]) is bridgeable. OCaml's type system is expressive enough to encode runtime invariants that dynamic languages can only check at runtime, enabling a unification of formal rigor and practical usability within a single toolchain.

8.3 Future Work

Several natural extensions follow from this work. Mechanized proofs in Coq, following the λ_A [4] methodology, would increase confidence in the LTS correctness proofs and monoid algebraic laws. Effect handlers for structured agent concurrency using OCaml 5 algebraic effects [8] would enable a more direct encoding of non-deterministic agent choice. A distributed variant of the state machine, with consensus-based transition validation, would support fault-tolerant multi-agent deployments. Extension to multi-agent coordination via the π-calculus offers a well-understood compositional semantics for inter-agent communication. Finally, Python and C bindings would enable embedded

deployment of the runtime in existing production environments that currently rely on less safe alternatives.

References

[1] Niclas A. H. Bahr and Casper Bach Poulsen. 2025. Effect Handlers for the Working Programmer. *arXiv preprint arXiv:2505.17167* (2025). arXiv:2505.17167 [cs.PL]

[2] Harrison Chase. 2022. LangChain: Building Applications with LLMs through Composability. https://github.com/langchain-ai/langchain Open-source framework.

[3] Simon Foldvik. 2026. Causal-Temporal Event Graphs: A Formal Model for Recursive Agent Execution Traces. *arXiv preprint arXiv:2604.17557* (2026). arXiv:2604.17557 [cs.LO]

[4] Qin Liu. 2026. λ_A: A Typed Lambda Calculus for LLM Agent Composition. *arXiv preprint arXiv:2604.11767* (2026). arXiv:2604.11767 [cs.PL]

[5] Microsoft. 2023. Semantic Kernel: An SDK to Integrate Large Language Models. https://github.com/microsoft/semantic-kernel Open-source SDK.

[6] OCaml Team. 2026. *The OCaml Manual, Release 5.4.* Inria. https://ocaml.org/manual/

[7] Andreas Schlapbach. 2026. Formal Semantics for Agentic Tool Protocols: A Process Calculus Approach. *arXiv preprint arXiv:2603.24747* (2026). arXiv:2603.24747 [cs.AI]

[8] K. C. Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, Anmol Sahoo, Sudha Parimala, Atul Dhiman, and Anil Madhavapeddy. 2021. Retrofitting Effect Handlers onto OCaml. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, Virtual, Canada, 206–221. doi:10.1145/3453483.3454039

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. Kigali, Rwanda. https://openreview.net/forum?id=tvI4u1ylcqs

[10] Simon Yu, Derek Chong, Ananjan Nandi, Dilara Soylu, Jiuding Sun, Christopher D. Manning, and Weiyan Shi. 2026. Shepherd: A Runtime Substrate Empowering Meta-Agents with a Formalized Execution Trace. *arXiv preprint arXiv:2605.10913* (2026). arXiv:2605.10913 [cs.AI]

[11] Yifan Zhang, Yang Yuan, Mengdi Wang, and Andrew Chi-Chih Yao. 2025. Monadic Context Engineering. *arXiv preprint arXiv:2512.22431* (2025). arXiv:2512.22431 [cs.AI]